

Individual Game Submission

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Adit Jigneshbhai Shah

Milestone 1 - Build Game Engine Foundations

Module 3 - Game Play Foundations

Individual Game Name: “Lost Under the Sea”

GitHub Game Repository:

<https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/ashah>

(A) Introduction

This document contains my individual implementation for Milestone 1 and explains how my game ‘Lost Under the Sea’ uses the shared team game engine. The milestone required not only a reusable engine structure but also an individual game showing the necessary systems, therefore, our team decided to divide reusable elements like window creation, rendering, entities, physics, input, collisions and scaling from the game logic. My game is a platformer where the player plays as a diver standing on a vent shelf in the same environment as the fish. The diver can move left/right, jump from any part of the platform, fall down and return back to the shelf when colliding with the fish. The game also satisfies scaling conditions.

(B) Shared Engine Architecture

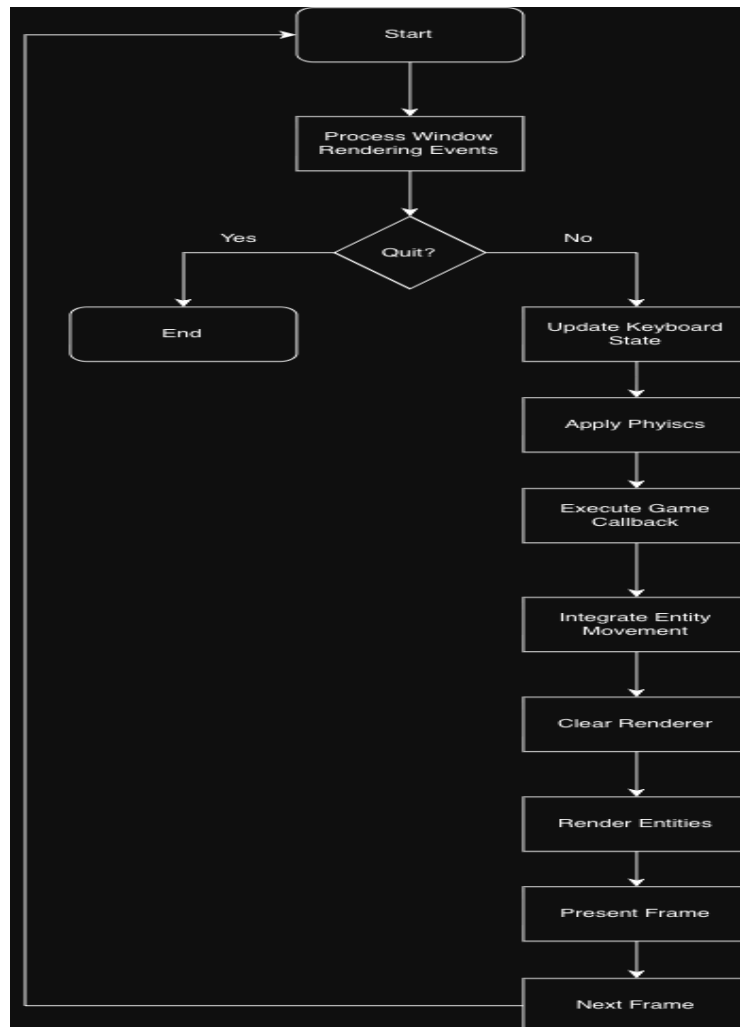
For Milestone 1, the team implemented a shared engine library.

The primary engine modules are:

- Engine
- Window
- Renderer

- Entity
- EntityManager
- Physics
- Input
- Collision
- Scaling

The Engine class is responsible for initiating SDL and executing the application loop. Every game uses this common library and provides its particular code in the respective main.cpp file. The main idea behind the engine design was to have the engine isolated from any kind of game-specific logic. This means that the engine itself does not implement any concrete game object such as Player, Enemy or Platform but rather implements the Entity with its generic attributes. It allows for the same engine interfaces to accommodate each individual's game without having to make any modifications for the games to the common library.



(fig. 1 - Main Frame Sequence)
(Made with the help of draw.io)

Some of the core design principles that are embodied by the shared architecture include the fact that gravity can be configured and used selectively, the fact that game inputs use keyboard-state polling instead of events, the separation of collision detection and collision handling, display scaling is done when rendering and not by changing the size of the simulation, and the inclusion of SDL3 as part of the project.

(C) Task 1 - Engine Initialization and Main Loop

(a) Implementation

My game constructs the shared Engine using the title *Engine engine("Lost Under the Sea")*. The engine initializes SDL, creates the application window and renderer, and executes the main loop through:

```
engine.run([&](float deltaTime) {  
    // game-specific update logic  
});
```

The game uses no second SDL event loop. All game activity for a single frame takes place inside the game callback that is passed to `Engine::run`. In every single frame, the engine shared among all the games performs:

1. quit and resize events handling
2. keyboard input update
3. physics
4. game callback execution
5. entity positions integration
6. renderer clearing
7. entity rendering
8. frame presentation

This means that the engine creates a full-featured continuously working game loop, and not just an SDL window. The real size of the SDL window, which cannot always be as large as 1920×1080 , is taken as the reference resolution for the game.



(fig. 2 - 'Lost Under the Sea' Game Screen)

(D) Task 2 - Required Game Entities

However, the abstraction called Entity is generic, whereas my game assigns roles for individual objects. My game includes all three entity types that the milestone specifies.

(a) Static Entity - Shelf

The shelf is generated using the shared entity mechanism with:

- velocity = 0
- gravity turned off
- sprite with texture

The shelf covers about 70% of the game's horizontal play area, leaving water empty space on both sides of it. This creates room for the player to walk past the platform and fall down due to gravity. Since the sprite contains some extra space above the actual floor of the shelf, there exists a different height of the shelf from the logical point of view.



(fig. 3 Static Entity - Shelf)
[Sprite generated with the help of ChatGPT Images]

(b) Controllable Entity - Diver

The diver is the controllable entity in the game. This entity is displayed using a four-frame sprite animation. Entity sizes are computed using the aspect ratio of frames so that the original artwork remains true to its intended size. Movements are programmed using the game's code and not the engine code.



(fig. 4 Controllable Entity - Diver)
[Sprite generated with the help of ChatGPT Images]

(c) Automatically Moving Entity - Fish

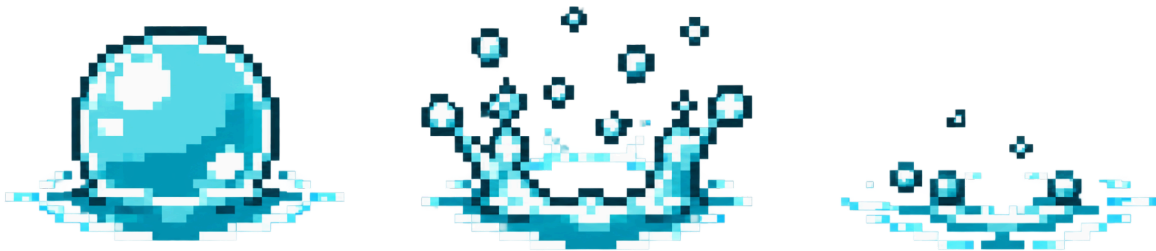
The fish moves horizontally along a pre-defined segment of the shelf indefinitely. The x-position of the fish is calculated by means of the following equation:

$$x += fishSpeed \times fishDirection \times deltaTime$$

Once the fish reaches the boundaries of its movement range, its direction is changed. For the sake of visualizing that change in direction, the flipHorizontal command is applied to the same 4 frame sprite image. This allows satisfying the requirements concerning an automatically moving entity without implementing any game-specific features within the common engine. Bubbles and ocean background images are included for visualization purposes only and are not counted as one of the three types of entities.



(fig. 5 Automatically Moving Entity - Fish)
[Sprite generated with the help of ChatGPT Images]



(fig. 6 Bubble Visualization)
[Sprite generated with the help of ChatGPT Images]

(E) Task 3 - Physics and Gravity

(a) Shared Physics Design

The gravity is implemented through a configurable acceleration in the Physics system that all entities share. For each entity that has the `affectedByGravity` flag set to true, the engine executes the following equation:

$$velocityY += gravity * deltaTime$$

This integration happens independently from the Position component and is done by the `EntityManager`.

(b) Game Configuration

For *Lost Under the Sea*, I configure gravity using `engine.getPhysics().setGravity(1600.0f)`. Only the diver is affected by gravity.

Entity	Gravity	Reason
Diver	Yes	Player must jump, fall, and land
Vent shelf	No	Static world geometry
Anglerfish	No	Scripted horizontal patrol
Background	No	Visual environment
Bubbles	No	Use independent upward motion

The jump animation is carried out using the upward velocity $velocityY = -720$. Following that, gravity from the engine pulls the character down continually. Should the character step over the shelf boundary to either side, there will be no special downward movement created in the game. The character falls because of gravity.



(fig. 7 Diver Resawning after colliding with fish)

(F) Task 4 - Keyboard Input and Player Controls

(a) Input Architecture

Gameplay Input is done via SDL keyboard state polling, not SDL keyboard events. `Input::update()` gets the current keyboard state using `SDL_GetKeyboardState(...)`. There are two main queries provided by the engine:

- `Input::isKeyPressed(...)`
- `Input::isKeyJustPressed(...)`

`isKeyPressed` is used for performing tasks repeatedly, like movement. `isKeyJustPressed` checks the state change from release to press and is used for discrete tasks like jumping and switching scale modes. Events of SDL are used only at the application level, for example for closing and resizing the window.

(b) Game Controls

Input	Action
A/Left Arrow	Move left
D/Right Arrow	Move right
W/Up Arrow/Space	Jump
Tab/T	Toggle Pixel/Percentage scaling

The horizontal movement will affect the speed of velocityX, enabling the use of the shared entity updating to incorporate the movement of the diver. The game does not affect the player's X position using the keyboard handler. Facing is achieved by the flipHorizontal method. An identical walking image strip will be used if the diver moves to the left.

(G) Task 5 - Collision Detection and Game Responses

(a) Shared Collision System

The shared engine provides:

```
Collision::checkCollision(  
    const Entity& a,  
    const Entity& b  
)
```

The function does Axis Aligned Bounding Box collision detection, returning a Boolean value. It is deliberate that the engine does not define a collision response. This ensures that each game decides what action to take following a collision detection. I have incorporated two main collision responses in my implementation.

(b) Diver-Shelf Collision

In the case that the diver intersects with the shelf during his descending motion, then the system will check if the player's center is inside the walkable region of the shelf. In case the conditions are met, then the diver will be placed on the logical walkable surface and his vertical velocity will become zero. This ensures that the player does not fall through the platform while enabling

the player to exit from either side. The logical walkway is different from the actual texture edge since the platform artwork includes coral and invisible padding above the visible edge.

(c) Diver–Fish Collision

The texture of both the diver and the fish has some level of transparency in it. Using the rectangles of the sprites as the hit boxes will result in collision before the two objects seem to touch each other. In order to make the collision visually correct, I have created inset body hit boxes for both entities. The collision function provided by the shared engine remains the same, but the rectangles being sent to it have been made smaller to more correctly resemble the actual bodies.

On the event of collision:

- The game prints the message that the fish has caught the diver
- The position of the diver on the X-axis becomes the spawn position
- The Y position becomes above the screen
- Velocities are reset
- The player falls down due to gravity

This collision can be seen without the need for a separate game over menu.

(d) Falling Beyond the Screen

The same resurrection rule applies to the scenario where the diver goes out of the platform and falls below the reference resolution. This is to ensure that the game keeps playing without interruption after the player exits the platform.

(H) Task 6 - Runtime Scaling

(a) Shared Scaling Design

The engine provides two display scaling modes:

ScaleMode::Pixel

ScaleMode::Percentage

In **Pixel mode**, entities are rendered according to their stored pixel dimensions. In **Percentage mode**, rendered positions and dimensions are multiplied by the ratio:

current window size / reference window size

The simulation values of the entity are not changed. This is significant because collision checking, physics, speed, and jumps remain relative to the world coordinates, despite any changes to the size of the rendered window.

(b) Game Demonstration

To toggle the scale mode, use Tab or T keys. Upon changing the scale mode:

- EntityManager::toggleScaleMode() switches the mode of rendering
- In percentage mode, the application window is resized to 65% of the reference window
- In pixel mode, the reference window size is restored
- The title bar is updated to reflect the current mode

The engine also restores the window size before resizing it under macOS since the maximized or zoomed window does not resize the window. Presentation through logic is set such that the resized window will be filled with rendered images and no more blue spaces.

(I) Design Limitations and Future Improvements

For Milestone 1, we have intentionally implemented light-weight systems suitable for the scope of the assignment. Collision detection uses inset AABB for now, rather than frame-by-frame or pixel-perfect hitboxes. Inset AABB suffices for now, although the next versions of the engine can implement customizable collision boxes per frame of animations. In the scaling demo, we have changed the size of the operating system window to demonstrate the difference between Pixel and Percentage scaling modes right away for the sake of grading. The future version of the system can implement the ability to dynamically resize the window, while retaining the rendering scale architecture. Currently, the engine doesn't implement advanced systems like rigid body physics, collision manifold, state machine for animations, and spatial partitioning. They have been intentionally left out for Milestone 1 to keep the shared game engine architecture minimal.

(J) Conclusion

Lost Under the Sea illustrates all of the Milestone 1 criteria through the use of the shared team engine. The engine offers reusable mechanisms for initializing the application, rendering, entities, physics, keyboard input, collision detection, and scaling. In my individual game, I use

the shared mechanisms to create a controllable diver, static platform, patrolling fish, jumping and falling logic, two different types of collisions, entity orientation, and dynamic scaling during runtime. The design makes sure there is an appropriate distinction between mechanisms offered by the shared library and policies implemented in the individual game. The shared library controls how entities are updated, drawn, overlapped, and how physics affect them, while the individual game controls what the entities are and what they do. This distinction allows for creating a reusable base for future milestones while meeting the criteria of Milestone 1.
